Recommendation Systems for Personalized Content Discovery

Technical Report

Cult Open Projects 2026

Netflix Prize Dataset

Prathamesh Barve – Rishabh Verma

Live Dashboard: netflix-recommender-frv5acygcv6v9ffixmjtvl.streamlit.app

1. Problem Understanding

Every day, billions of users interact with recommendation systems that shape the content they consume. The Netflix Prize — one of the most influential machine learning competitions in history — challenged teams worldwide to improve Netflix's recommendation algorithm, producing a benchmark dataset that continues to drive research.

In this project, we step into the role of Machine Learning Engineers tasked with building an intelligent recommendation engine on the Netflix Prize Dataset. The core challenge is not simply to predict ratings accurately, but to deliver meaningful, personalized content recommendations that genuinely reflect user preferences.

The key technical challenges include:

- Extreme data sparsity — most users have rated only a tiny fraction of available movies
- Long-tail user activity — a small fraction of power users dominate the interaction data
- Cold start problem — new or sparse users have insufficient history for high-quality recommendations
- Scale — 100M+ ratings across 480K users and 17,770 movies

2. Exploratory Data Analysis

2.1 Dataset Overview

Due to computational constraints in Google Colab, we worked with a representative sample of 500,000 ratings from combined_data_1.txt — the first of four data files. After filtering for active users (≥10 ratings) and popular movies (≥20 ratings), our working dataset contained:

Metric	Value
Total Ratings (sample)	43,224
Unique Users	3,203
Unique Movies	1,317
Total Movies (full catalog)	17,770
Rating Scale	1 to 5 stars
Data Sparsity	~99%
Train Set Size	34,579 ratings
Test Set Size	8,645 ratings

2.2 Rating Distribution

The rating distribution is strongly left-skewed, with approximately 55% of all ratings being 4 or 5 stars. This reflects a well-known selection bias in user-generated ratings — users tend to rate content they have already chosen to watch, which they are more likely to enjoy.

- Mean rating: ~3.6 stars
- Median rating: 4 stars
- This validates our choice of 3.5 stars as the MAP@10 relevance threshold — it lies close to the median and separates genuinely liked content from neutral/negative ratings
- Business implication: A recommendation system can achieve reasonable user satisfaction by consistently predicting 3.5+ star content

2.3 User Activity Patterns

User activity follows a classic power law distribution:

- Mean ratings per user: 13.5
- Median ratings per user: 12
- Maximum ratings by a single user: 143
- Minimum (post-filter): 10

The highly skewed distribution means a small percentage of active users account for a disproportionate share of ratings. This has two implications: (1) collaborative filtering will work well for active users, and (2) sparse users face a cold start problem where recommendations are less reliable.

2.4 Content Popularity

Similarly, movie popularity follows a long-tail distribution. A handful of well-known titles (Lord of the Rings series, classic TV shows, animated films) attract the vast majority of ratings, while thousands of niche titles have very few interactions. This reinforces the need for personalized recommendations — a pure popularity-based system would recommend the same top-10 movies to every user regardless of individual taste.

2.5 Data Sparsity

The user-movie interaction matrix has approximately 99% sparsity — meaning each user has rated less than 1% of available movies. This is the defining challenge of recommendation systems and motivates latent factor models like SVD, which learn compressed representations that generalize well despite missing data.

3. Methodology

3.1 Data Preparation

The Netflix Prize data is distributed in a custom format where each file contains a movie_id header line followed by user_id, rating, date triples. We wrote a custom parser to handle this format, then applied the following filters:

- Minimum 10 ratings per user — ensures sufficient preference signal
- Minimum 20 ratings per movie — ensures reliable item statistics
- Random sample of 500K rows from file 1 — manages Colab memory constraints

The problem statement explicitly permits working with subsets of the data for computational reasons.

3.2 Train-Test Split

We used an 80/20 random train-test split with random_state=42 for full reproducibility. This resulted in 34,579 training interactions and 8,645 test interactions. The random split ensures the test set contains a representative sample of users and movies seen during training.

3.3 Relevance Definition for MAP@10

Following the problem statement specification, a movie is defined as relevant if its actual user rating is greater than or equal to 3.5 stars. This threshold was chosen because it lies above the dataset mean (3.6) and effectively separates content users found worthwhile from content they were neutral or negative about.

3.4 MAP@10 Computation Procedure

For each user in the test set, we: (1) identify all movies in the dataset not rated by that user in the training set, (2) generate predicted ratings for all such movies using the trained model, (3) rank them by predicted rating in descending order, (4) take the top 10, and (5) check how many have actual ratings ≥3.5 in the test set. Average Precision is computed per user and averaged across all users to yield MAP@10.

4. Model Design

4.1 Model 1 — SVD (Matrix Factorization)

Singular Value Decomposition (SVD) decomposes the user-movie rating matrix R into latent factor matrices: R ≈ P × Q^T, where P represents user preference vectors and Q represents movie characteristic vectors in a shared latent space.

Configuration used:

- 100 latent factors — captures nuanced taste dimensions such as genre affinity, era preference, and pacing
- 20 training epochs with stochastic gradient descent
- Learning rate: 0.005 | Regularization: 0.02

SVD was chosen as the primary model because it was the foundational technique behind the winning Netflix Prize submission. It handles extreme sparsity well by learning global patterns across all users and movies simultaneously, rather than relying on local similarity.

4.2 Model 2 — User-Based Collaborative Filtering

User-Based CF (KNNBasic) finds the k most similar users to the target user (using cosine similarity) and recommends movies they rated highly that the target user has not yet seen.

Configuration used:

- k = 40 nearest neighbors
- Cosine similarity metric
- User-based (not item-based)

CF provides an intuitive, explainable approach — recommendations can be justified as 'users like you also enjoyed this'. However, it is computationally expensive at scale and struggles with sparse users who have few neighbors with sufficient overlap.

4.3 Similar Movies — Latent Factor Similarity

Beyond rating prediction, we implemented an item-item similarity module using the SVD latent factors. For any target movie, we compute the cosine similarity between its learned latent vector (Q[i]) and all other movies' latent vectors. This enables the 'Find Similar Movies' feature in our interactive dashboard without any additional training.

This also serves as the Explainable AI component — the system can surface movies that occupy similar positions in the learned latent space, providing an interpretable basis for recommendations.

5. Evaluation Metrics & Results

5.1 Results Summary

Model	RMSE	MAE	MAP@10
SVD (Matrix Factorization)	0.9677	0.7663	0.8514
User-Based Collaborative Filtering	1.0977	0.8820	0.8441

5.2 RMSE Analysis

RMSE measures average rating prediction error. SVD achieves an RMSE of 0.9677, meaning its predicted ratings deviate from actual ratings by less than 1 star on average on a 1–5 scale. This is a strong result and consistent with published SVD benchmarks on the Netflix Prize dataset.

User-Based CF achieves RMSE of 1.0977 — meaningfully higher than SVD. This is expected because CF relies on finding similar users, which becomes unreliable in a sparse dataset where user overlap is limited.

5.3 MAP@10 Analysis

MAP@10 evaluates recommendation ranking quality — whether the truly relevant movies appear near the top of the recommendation list. Both models achieve impressive MAP@10 scores:

- SVD: 0.8514 — meaning on average, 85% of the top-10 recommendations are movies the user actually rated ≥3.5 stars
- User-Based CF: 0.8441 — very close to SVD on ranking quality despite higher RMSE

The high MAP@10 scores for both models indicate that both are effective at surfacing content users genuinely enjoy, even if their exact predicted ratings differ. This is ultimately the more important metric for a production recommendation system.

5.4 Trade-off Discussion

There is a subtle but important distinction between RMSE and MAP@10. RMSE penalizes all prediction errors equally, while MAP@10 only cares whether relevant items appear at the top of the list. A model can have relatively high RMSE but still achieve high MAP@10 if it correctly ranks relevant items above irrelevant ones — this is exactly what we observe with User-Based CF.

For a production recommendation system, MAP@10 is the more business-relevant metric since users care about what appears in their top recommendations, not about exact predicted star ratings.

6. Recommendation Examples

6.1 Success Case — Active User (ID: 1933317)

This user has rated 5 movies in our sample, all with 5-star ratings including Star Trek: Voyager, Justice League, and Lilo and Stitch — indicating strong affinity for sci-fi, animation, and adventure content.

SVD Top-5 Recommendations:

Rank	Recommended Movie	Predicted Rating
1	Lord of the Rings: Return of the King (Extended)	4.54
2	The Game	4.42
3	Magnolia: Bonus Material	4.40
4	Dragonheart	4.35
5	ABC Primetime: Mel Gibson's The Passion of the Christ	4.30

The model correctly identifies high-confidence recommendations (4.3–4.5 range) that align with the user's demonstrated preference for epic/adventure content. This is a strong success case.

6.2 Failure Case — Sparse User (ID: 1982465)

This user has no movies rated ≥4 stars in the sampled portion of the dataset, indicating either a very critical rater or a user whose preferences are poorly represented in our sample. The model generates recommendations in the 3.3–3.7 range — noticeably lower confidence than for active users.

This demonstrates the cold start problem: when a user's preference profile is sparse or unclear, the latent factor model cannot precisely locate them in the preference space, resulting in more generic and less confident recommendations.

Mitigation strategy: For cold start users, a popularity-based fallback (recommending the highest-rated movies overall) would provide better initial recommendations until sufficient interaction history is accumulated.

6.3 Key Observations

- Recommendation confidence (predicted rating) scales directly with user activity — more ratings = higher and more personalized predictions
- SVD latent factors correctly capture abstract preference dimensions — users who like animation also get animated recommendations even without explicit genre labels
- The similar movies feature (latent factor cosine similarity) surfaces thematically related content that shares latent characteristics, providing an explainable basis for recommendations

7. Interactive Dashboard

As part of Optional Tasks C and E, we built and deployed a fully functional recommendation dashboard using Streamlit. The live application is accessible at:

https://netflix-recommender-frv5acygcv6v9ffixmjtvl.streamlit.app

The dashboard consists of four pages:

- Home — Dataset statistics (total ratings, users, movies, sparsity) and EDA visualizations including rating distribution and user activity histogram
- User Recommendations — Enter any User ID to view their taste profile (movies rated, average rating, top-rated titles) alongside SVD Top-10 personalized recommendations color-coded by confidence
- Similar Movies — Search any movie title to find the 10 most similar movies using cosine similarity on SVD latent factors — this serves as the Explainable AI component
- Model Performance — Side-by-side RMSE and MAP@10 comparison charts with full methodology documentation

8. Key Insights

- SVD consistently outperforms User-Based CF on both RMSE (0.9677 vs 1.0977) and MAP@10 (0.8514 vs 0.8441), confirming that latent factor models are better suited to sparse, large-scale recommendation problems
- Both models achieve MAP@10 above 0.84, demonstrating that even with a 500K row sample, collaborative filtering approaches can deliver highly relevant recommendations
- Data sparsity (~99%) is the central challenge — models that learn global patterns (SVD) handle this better than local similarity methods (CF)
- Recommendation quality is strongly tied to user activity — the cold start problem remains unsolved for sparse users in a pure collaborative filtering framework
- Latent factor similarity provides a computationally free explainability mechanism — SVD's qi vectors directly encode item relationships that can be surfaced to users
- The MAP@10 vs RMSE trade-off shows that optimizing for ranking quality is more business-relevant than minimizing prediction error

9. Future Improvements

- Train on the full 100M rating dataset using distributed computing for significantly better model generalization
- Implement SVD++ which incorporates implicit feedback (which movies a user rated, regardless of the score) for improved cold start handling
- Add a Neural Collaborative Filtering (NCF) model using deep learning to capture non-linear user-item interactions
- Implement a hybrid system combining SVD latent factors with content-based features (movie genre, release year, director) to improve recommendations for sparse users
- Deploy with a real-time re-ranking layer that applies business rules (e.g. diversity constraints, freshness boosting) on top of the base model scores
- Implement A/B testing framework to measure actual user engagement with different recommendation strategies